МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №9
з дисципліни «Професійний Python»

Тема: «Паралельність, багатопроцесність та оптимізація»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Дослідити вплив GIL на CPU-bound та I/O-bound задачі, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor, знаходити bottleneck за допомогою cProfile, timeit/perf_counter і tracemalloc, застосовувати NumPy-векторизацію та кешування і експериментально визначати, який спосіб оптимізації справді прискорює конкретну операцію проєкту.

1. Постановка задачі, проєкт і тестові дані

Варіант 4. Система банківських рахунків. Створити 100 000+ рахунків і 500 000+ операцій, профілювати загальну суму коштів, оборот, статистику операцій і розрахунок балансу за великою історією; порівняти Python loops, NumPy та ProcessPoolExecutor; паралельно читати журнали операцій через ThreadPoolExecutor; змоделювати конкурентні поповнення/списання у спільній пам'яті з Lock. Проєкт продовжує ЛР1–ЛР8: правило ЛР1 (списання понад баланс відхиляється) і гроші в цілих копійках (ЛР7) збережено, тому результати всіх реалізацій порівнюються точно.

Досліджувана операція – пакетна обробка «кінця періоду» `end_of_day`: перерахунок балансів за всією історією з підсумковим звітом (загальна сума, оборот, кількість відхилених списань, середнє, стандартне відхилення і максимум сум) та нарахування 10 % річних зі щоденною капіталізацією й округленням до копійки (365 кроків на рахунок).

Таблиця 1 – Тестові дані (generate, seed = 42) та обладнання

Набір	Рахунки	Операції	Характеристики
малий	10 000	50 000	суми й баланси – логнормальний розподіл (медіана балансу ≈ 7 300 грн, операції ≈ 300 грн); 55 % поповнень, 45 % списань; 1.7% списань (0.76% усіх операцій) перевищують баланс і відхиляються
основний	100 000	500 000	той самий генератор
великий	400 000	2 000 000	той самий генератор
ПК	–	–	Intel Core i7-14650HX (16 ядер: 8 P + 8 E, 24 потоки), 16 ГБ, Windows 11, Python 3.12.10, NumPy 2.5.4

```
Lab_09/src/bank_perf/
├── analytics.py    baseline (Python loops): report_python, accrue_interest; шарди і злиття
├── parallel.py     ThreadPoolExecutor, ProcessPoolExecutor, multiprocessing.Process + Queue
├── vectorized.py   NumPy: report_numpy, accrue_numpy
├── journals.py     CSV-журнали, послідовне і паралельне читання (I/O-bound)
├── shared.py       спільний рахунок у потоках: race condition і Lock
├── ledger.py       lru_cache виписок рахунку з інвалідацією
├── profiling.py    cProfile, tracemalloc, perf_counter;  cli.py - bank-perf
benchmarks/benchmark.py   повтори, speedup, перевірка результатів -> results/*.json, *.csv
```

2. Baseline і профілювання (cProfile)

```
def apply_operation(balances: list[int], account: int, kind: int, amount: int) -> bool:
    if kind == WITHDRAW:
        if amount > balances[account]:
            return False
        balances[account] -= amount
    else:
        balances[account] += amount
    return True
```

```python
def accrue_one(balance: int, days: int = DAYS) -> int:
    for _ in range(days):
        balance += (balance + HALF) // DAILY_DIVISOR
    return balance

def report_python(opening: Sequence[int], operations: Iterable[Operation]) -> LedgerReport:
    balances = list(opening)
    accepted: list[int] = []
    deposits = withdrawals = rejected = 0
    for account, kind, amount in operations:
        if apply_operation(balances, account, kind, amount):
            accepted.append(amount)
            if kind == DEPOSIT:
                deposits += amount
            else:
                withdrawals += amount
        else:
            rejected += 1
    mean, std, maximum = amount_statistics(accepted)
    return LedgerReport(balances, total_funds(balances), deposits, withdrawals, len(accepted),
                        rejected, mean, std, maximum)
```

```
PS C:\Github\prof_python\Lab_09> bank-perf profile
Дані: 100 000 рахунків, 500 000 операцій
         1592444 function calls in 1.935 seconds

   Ordered by: cumulative time
   List reduced from 15 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.003    0.003    1.935    1.935 analytics.py:114(end_of_day)
        1    0.038    0.038    1.549    1.549 analytics.py:104(accrue_interest)
   100000    1.512    0.000    1.512    0.000 analytics.py:97(accrue_one)
        1    0.125    0.125    0.383    0.383 analytics.py:78(report_python)
   500000    0.107    0.000    0.107    0.000 analytics.py:50(apply_operation)
        1    0.000    0.000    0.102    0.102 analytics.py:68(amount_statistics)
        2    0.034    0.017    0.098    0.049 {built-in method builtins.sum}
   496216    0.063    0.000    0.063    0.000 analytics.py:74(<genexpr>)
   496215    0.044    0.000    0.044    0.000 {method 'append' of 'list' objects}
        1    0.005    0.005    0.005    0.005 analytics.py:61(total_funds)
Bottleneck (найбільший tottime): accrue_one - 100 000 викликів, 1.51 с з 1.94 с (78%)
```

Рисунок 1 – Профіль end_of_day на основному наборі

`ncalls` показує, що `accrue_one` викликається 100 000 разів (раз на рахунок), а `apply_operation` – 500 000 разів (раз на операцію). `tottime` – власний час функції без вкладених викликів, `cumtime` – разом із ними. Bottleneck – `accrue_one`: 1.51 с власного часу з 1.94 с (78 %); перерахунок історії (`report_python`, cumtime 0.383 с) – другий за вагою. Задача CPU-bound: лише цілочисельна арифметика в циклах, без очікування I/O. cProfile сам додає накладні витрати на кожен виклик, тому абсолютні часи під профайлером більші, ніж у benchmark.

3. Пам'ять (tracemalloc)

```
PS C:\Github\prof_python\Lab_09> bank-perf memory
Журнал 500 000 операцій + 100 000 рахунків
  список кортежів Python:    66.9 MB
  масиви NumPy:               8.9 MB
Пік пам'яті під час обчислення (tracemalloc):
  report_python    пік    7.8 MB, лишилось   3.8 MB
  report_numpy     пік   31.7 MB, лишилось   0.8 MB
  accrue_interest  пік    3.8 MB, лишилось   3.8 MB
  accrue_numpy     пік    2.3 MB, лишилось   0.8 MB
Найбільші алокації report_python:
  analytics.py:57: size=1723 KiB, count=55123, average=32 B
  analytics.py:55: size=1381 KiB, count=44184, average=32 B
  analytics.py:80: size=781 KiB, count=2, average=391 KiB
```

Рисунок 2 – Пам'ять журналу та пік під час обчислень

Список із 500 000 кортежів займає 66.9 МБ: кожна операція – окремий об'єкт-кортеж і об'єкти int. Ті самі дані в трьох стовпцях NumPy – 8.9 МБ (у

7.5 раза менше). Водночас пік векторизованого звіту більший (31.7 МБ проти 7.8 МБ): NumPy створює тимчасові масиви (сортування, cumsum, маски). Найбільше пам'яті в baseline виділяють нові int-об'єкти балансів у `apply_operation`.

4. Реалізації оптимізацій

Threads і processes. Рахунки незалежні, тож роботу ділимо за рахунками: для відсотків – на порції балансів, для звіту – на шарди `account % N` (порядок операцій кожного рахунку зберігається, частини звіту зливаються з точних цілих сум). Функції воркерів лежать у модулі без NumPy, щоб процеси стартували швидше; у Windows процеси запускаються через spawn, тому код запуску – під `if __name__ == "__main__"`.

```
def accrue_threads(balances: Sequence[int], workers: int, days: int = DAYS) -> list[int]:
    with ThreadPoolExecutor(max_workers=workers) as executor:
        parts = executor.map(partial(accrue_chunk, days=days), split(balances, workers))
        return [balance for part in parts for balance in part]

def accrue_processes(
    balances: Sequence[int], workers: int, days: int = DAYS, chunksize: int | None = None
) -> list[int]:
    chunk = chunksize or max(1, len(balances) // (workers * 4))
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(partial(accrue_one, days=days), balances, chunksize=chunk))

def report_multiprocessing(
    opening: Sequence[int], operations: Sequence[Operation], workers: int
) -> LedgerReport:
    openings, parts = _shards(opening, operations, workers)
    results: Queue[tuple[int, Partial]] = multiprocessing.Queue()
    processes = [
        multiprocessing.Process(target=_process_worker, args=(shard, openings[shard],
                                                              parts[shard], results))
        for shard in range(workers)
    ]
    for process in processes:
        process.start()
    # Спершу забрати всі результати: процес не завершиться, поки його дані в черзі не прочитано.
    collected = dict(results.get() for _ in processes)
    for process in processes:
        process.join()
    return merge(opening, [collected[shard] for shard in range(workers)])
```

NumPy. Відсотки рахуються 365 векторними кроками для всіх рахунків разом. Звіт векторизується так: після стабільного сортування за рахунком `cumsum` дає поточний баланс після кожної операції; рахунки, де він ніде не став від'ємним (97% рахунків з операціями), не мають відхилених списань, і їхній результат готовий; решту (3.8% операцій) перераховує короткий Python-цикл.

```
def accrue_numpy(balances: Int64Array, days: int = DAYS) -> Int64Array:
    result = balances.copy()
    for _ in range(days):
        result += (result + HALF) // DAILY_DIVISOR
    return result

# report_numpy (фрагмент)
    accounts_sorted = data.account[order]
    signed_sorted = signed[order]
    starts = np.flatnonzero(np.r_[True, accounts_sorted[1:] != accounts_sorted[:-1]])
    lengths = np.diff(np.r_[starts, len(order)])
    group_accounts = accounts_sorted[starts]

    # running-баланс у межах кожного рахунку: загальний cumsum мінус суму до початку групи
    running = np.cumsum(signed_sorted)
    before_group = running[starts] - signed_sorted[starts]
    running += np.repeat(data.opening[group_accounts] - before_group, lengths)

    balances = data.opening.copy()
```

```
    balances[group_accounts] = running[starts + lengths - 1]
    accepted = np.ones(len(order), dtype=bool)
    troubled = np.minimum.reduceat(running, starts) < 0
    positions = order[np.repeat(troubled, lengths)]  # операції «проблемних» рахунків, по порядку
    if len(positions):
        _replay(positions, data, balances, accepted)
```

Synchronization і caching. Списання – «перевір, потім зміни», тому читання, перевірки і запис балансу утворюють критичну секцію під Lock. Виписка рахунку проходить увесь журнал, тож повторні запити кешуються `lru_cache`; будь-яка прийнята операція очищає кеш (cache invalidation), бо виписки стають застарілими.

```
# class SharedAccount (shared.py)
def apply(self, kind: int, amount: int) -> bool:
    with self._lock or nullcontext():  # критична секція: читання - перевірки - запис
        current = self.balance
        if not self._within_limit(amount):
            return False
        if kind == WITHDRAW:
            if amount > current:
                return False
            current -= amount
        else:
            current += amount
        self.balance = current
        return True

# class Ledger (ledger.py): self.statement = lru_cache(maxsize=1024)(self._statement)
def apply(self, account: int, kind: int, amount: int) -> bool:
    accepted = self._apply(account, kind, amount)
    if accepted:
        self.statement.cache_clear()  # cache invalidation
    return accepted
```

5. Результати benchmark

Кожен метод запускався 5 разів (`perf_counter`), повтори методів чергувалися; перед вимірюванням кожен результат порівнювався з baseline. Speedup = середній час baseline / середній час методу; Threads і Processes – 8 workers.

Таблиця 2 – Нарахування відсотків, 100 000 рахунків: час окремих запусків, с

Метод	Run 1	Run 2	Run 3	Run 4	Run 5	Середнє	Speedup
Python	1.449	1.467	1.454	1.464	1.456	**1.458**	1.00×
Threads	1.415	1.553	1.395	1.395	1.420	**1.436**	1.02×
Processes	0.587	0.555	0.567	0.557	0.583	**0.570**	2.56×
NumPy	0.184	0.183	0.202	0.175	0.184	**0.186**	7.85×

Таблиця 3 – Нарахування відсотків для різних розмірів: середній час, с (speedup)

Рахунки	Python	Threads	Processes	NumPy
10 000	0.147 (1.00×)	0.143 (1.03×)	0.347 (0.42×)	0.006 (24.49×)
100 000	1.458 (1.00×)	1.436 (1.02×)	0.570 (2.56×)	0.186 (7.85×)
400 000	5.869 (1.00×)	5.918 (0.99×)	1.357 (4.33×)	0.709 (8.28×)

Таблиця 4 – Кількість workers (100 000 рахунків): середній час, с (speedup)

Workers	1	2	4	8	16
ThreadPoolExecutor	1.532 (0.99×)	1.478 (1.03×)	1.487 (1.02×)	1.493 (1.02×)	1.524 (1.00×)
ProcessPoolExecutor	1.694 (0.90×)	0.980 (1.56×)	0.666 (2.29×)	0.555 (2.75×)	0.927 (1.64×)

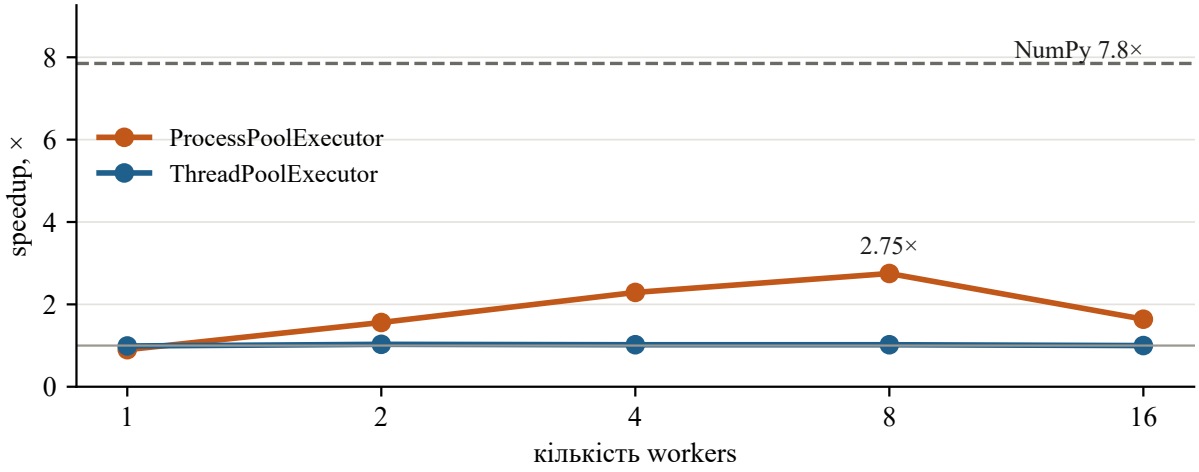


Рисунок 3 – Speedup нарахування відсотків залежно від кількості workers

Таблиця 5 – Звіт за журналом (перерахунок балансів, сума, оборот, статистика): середній час, с (speedup)

Операцій	Python	Threads	Processes	Process + Queue	NumPy
50 000	0.012 (1.00×)	0.017 (0.69×)	0.331 (0.04×)	1.601 (0.01×)	0.006 (2.01×)
500 000	0.151 (1.00×)	0.197 (0.76×)	0.446 (0.34×)	1.773 (0.08×)	0.073 (2.07×)
2 000 000	0.801 (1.00×)	0.976 (0.82×)	1.055 (0.76×)	2.304 (0.35×)	0.345 (2.32×)

Таблиця 6 – Інші експерименти

Експеримент	Результат
chunksize (10 000 рахунків, 8 процесів)	Python: 0.15 с; chunksize 1: 1.62 с; chunksize 10: 0.43 с; chunksize 100: 0.33 с; chunksize 1000: 0.34 с
Читання 8 CSV-журналів (500 000 операцій)	Sequential: 0.425 с (1.00×); Threads: 0.397 с (1.07×); Processes: 0.449 с (0.95×)
Те саме + затримка сховища 0.1 с на файл	Sequential: 1.226 с (1.00×); Threads: 0.487 с (2.52×); Processes: 0.543 с (2.26×)
Race condition (8 потоків × 20 000 операцій, квант 1 мкс)	втрачено без Lock: 61 917.16, 50 436.57, 196 769.82, 143 492.33, 139 602.18 грн; з Lock: 0.00, 0.00, 0.00, 0.00, 0.00 грн
Накладні витрати Lock (звичайний квант 5 мс)	без Lock: 0.122 с; з Lock: 0.107 с
lru_cache виписки (100 000 рахунків)	перший виклик 10.16 мс, повторний 0.05 мкс; 500 запитів: 377 hits / 123 misses, 1.45 с проти 5.85 с без кешу

```
PS C:\Github\prof_python\Lab_09> bank-perf race
8 потоків × 20 000 операцій над одним рахунком (switchinterval 1 мкс)
  без Lock: баланс 124 266.33 UAH, за журналом прийнятих операцій 284 015.92 UAH, розбіжність 159 749.59 UAH,
1.59 с
  з Lock:   баланс 284 015.92 UAH, за журналом прийнятих операцій 284 015.92 UAH, розбіжність 0.00 UAH, 0.74 с
PS C:\Github\prof_python\Lab_09> bank-perf cache
statement(42): перший виклик 11.3 мс, повторний 0.05 мкс (у 231 239 разів швидше)
1000 запитів виписок (80 % - до 20 популярних рахунків): 2.57 с, CacheInfo(hits=780, misses=220, maxsize=1024,
currsize=220)
Після поповнення рахунку 42 на 100.00 UAH кеш очищено: баланс 12 931.90 UAH -> 13 031.90 UAH, операцій 5 -> 6,
CacheInfo(hits=0, misses=1, maxsize=1024, currsize=1)
```

Рисунок 4 – Race condition і Lock; кешування виписок та інвалідація

6. Перевірка правильності та тести

16 тестів перевіряють, що всі реалізації звіту (потоки, процеси з 1, 3 і 8 workers, Process + Queue, NumPy) і відсотків дають той самий результат, що й

baseline: баланси й суми в копійках – точно, середнє і стандартне відхилення – з відносним допуском 10^{-9}. Нарахування відсотків звірено з незалежною реалізацією на `Decimal` з `ROUND_HALF_UP`. Окремо перевірено правило ЛР1, Lock (розбіжність 0), виявлення гонки без Lock, hit/miss та інвалідацію кешу, порядок операцій після паралельного читання журналів.

```
PS C:\Github\prof_python\Lab_09> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 --
C:\Github\prof_python\Lab_09\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Github\prof_python\Lab_09
configfile: pyproject.toml
testpaths: tests
collecting ... collected 16 items

tests/test_concurrency_and_cache.py::test_lock_keeps_balance_consistent PASSED [  6%]
tests/test_concurrency_and_cache.py::test_race_condition_without_lock_is_detectable PASSED [ 12%]
tests/test_concurrency_and_cache.py::test_statement_cache_hits_and_invalidation PASSED [ 18%]
tests/test_concurrency_and_cache.py::test_journal_loaders_return_operations_in_order[1] PASSED [ 25%]
tests/test_concurrency_and_cache.py::test_journal_loaders_return_operations_in_order[4] PASSED [ 31%]
tests/test_correctness.py::test_baseline_follows_lab1_rules PASSED       [ 37%]
tests/test_correctness.py::test_dataset_has_overdraft_attempts PASSED    [ 43%]
tests/test_correctness.py::test_thread_and_process_reports_match_baseline[1] PASSED [ 50%]
tests/test_correctness.py::test_thread_and_process_reports_match_baseline[3] PASSED [ 56%]
tests/test_correctness.py::test_thread_and_process_reports_match_baseline[8] PASSED [ 62%]
tests/test_correctness.py::test_multiprocessing_report_matches_baseline PASSED [ 68%]
tests/test_correctness.py::test_numpy_report_matches_baseline PASSED     [ 75%]
tests/test_correctness.py::test_numpy_report_without_operations PASSED   [ 81%]
tests/test_correctness.py::test_interest_rounds_to_cents_daily PASSED    [ 87%]
tests/test_correctness.py::test_interest_versions_match_baseline[1] PASSED [ 93%]
tests/test_correctness.py::test_interest_versions_match_baseline[4] PASSED [100%]

============================= 16 passed in 8.63s ==============================
PS C:\Github\prof_python\Lab_09> mypy
Success: no issues found in 13 source files
PS C:\Github\prof_python\Lab_09> ruff check .
All checks passed!
```

Рисунок 5 – Результат pytest -v (16 тестів за 8.63 с), mypy --strict і ruff

7. Аналіз результатів

Bottleneck – нарахування відсотків (`accrue_one`, 78% часу під профайлером), задача CPU-bound. **ThreadPoolExecutor** не допоміг за жодної кількості потоків (1.02× з 8): через GIL Python-байткод потоків виконується по черзі. **ProcessPoolExecutor** залежить від обсягу роботи: на 10 000 рахунків він повільніший за baseline (0.42× – запуск 8 процесів дорожчий за 0.15 с обчислень), на 100 000 – 2.56×, на 400 000 – 4.33×. Оптимум – 8 workers (2.75×); з 16 прискорення падає до 1.64×: старт кожного процесу (spawn) коштує більше, ніж дає додатковий паралелізм, до того ж 8 з 16 ядер цього CPU – повільніші E-ядра. `chunksize = 1` (рахунок за повідомлення) у 4.8 раза повільніший за `chunksize = 100`. Найкращий результат – **NumPy**: 7.8× на 100 000 і 8.3× на 400 000 рахунків, бо 365 кроків виконуються для всіх рахунків одночасно в скомпільованому коді; пам'яті при цьому потрібно менше.

Для **звіту за журналом** процеси повільніші на всіх розмірах (до 0.76× на 2 000 000 операцій): обробка операції дешева, і серіалізація сотень тисяч кортежів коштує більше за саму роботу. Process + Queue ще повільніший: у Windows `start()` передає дані шарду через канал запуску і чекає, доки дочірній процес (після імпорту головного модуля) їх прочитає, тож процеси стартують по черзі; ProcessPoolExecutor спершу паралельно запускає

порожніх воркерів. NumPy швидший у 2.07 раза, але лише якщо дані вже в масивах: перетворення списку займає 0.089 с, і разом виходить повільніше за baseline – масиви варто формувати одразу під час завантаження журналу. Для **I/O-bound** читання потоки ефективні при справжньому очікуванні (2.52× із затримкою сховища); з локального SSD час визначає розбір CSV (CPU), і потоки дають лише 1.07×. **Lock** усунув втрати й не додав помітних витрат (0.107 с проти 0.122 с без нього). **Caching** скоротив 500 запитів виписок з 5.85 до 1.45 с, але коректний лише з інвалідацією після кожної зміни. Для production доцільно залишити NumPy для відсотків і статистики, кеш виписок з інвалідацією і Lock для спільного стану.

Висновки

Для банківської системи (варіант 4) на даних до 400 000 рахунків і 2 000 000 операцій виконано профілювання (cProfile, tracemalloc), визначено bottleneck і порівняно baseline, ThreadPoolExecutor, ProcessPoolExecutor, multiprocessing.Process, NumPy та кешування з 5 повторами для 3 розмірів даних і 1–16 workers. Для CPU-bound нарахування відсотків отримано прискорення 7.8–8.3× (NumPy) і 2.56–4.33× (процеси, 100 000–400 000 рахунків), потоки його не прискорили; потоки корисні лише для очікування I/O, процеси – лише коли обчислення дорожчі за передачу даних. Lock усунув race condition, кеш – повторні обчислення. Правильність усіх реалізацій підтверджено 16 тестів.